Proyecto Integrador 3

MAESTRO:

LOPEZ PIÑA DANIEL

MATERIA:

DISEÑO ELECTRONICO BASADO EN SISTEMAS EMBEBIDOS

EQUIPO:

Benavides Villanueva Bernabe Manuel

Calderón Maldonado Jair Alejandro

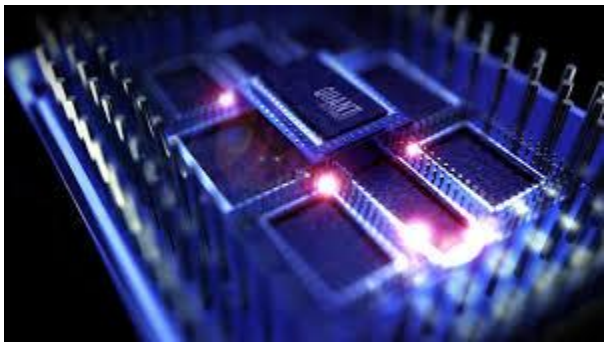
Castelan Gamoneda Isaac

Romany Bustos Joshua

Rivera González Luis Mario

Pérez Bautista Luis Manuel

López Camacho Jonathan Valentín



Indice

Proyecto Integrador 3 1
Lista De Materiales 3
Diagrama De Estados 4
Esquema De Conexión 6
Circuito Montado 8
SKETCH 12
Al ingreso de solo 3 dígitos 19
PRUEBAS 19
Conclusiones 20

ETAPA 1	Estudiantes 1 y 2 — Analistas de Requisitos
Tareas a realizar	• Analizar los 7 requisitos e identificar las máquinas de estados necesarias • Definir el código de acceso de ejemplo (ej: 1-3-2-4) • Listar estados del sistema: ESPERA, INGRESANDO, VERIFICANDO, ABIERTO, BLOQUEADO_CORTO, BLOQUEADO_LARGO • Identificar componentes y pines necesarios • Establecer el mapa de pines completo
Entregables	• **Documento de análisis con diagrama de estados (puede ser a mano)** • **Lista de materiales y mapa de pines**
Criterios de aceptación	• Los 6 estados del sistema están definidos • Las transiciones entre estados están especificadas

Mapa de pines

1. Botón B1 (dígito 1) 2 Pull-down externo 10kΩ
2. Botón B2 (dígito 2) 3 Pull-down externo 10kΩ
3. Botón B3 (dígito 3) 4 Pull-down externo 10kΩ
4. Botón B4 (dígito 4) 5 Pull-down externo 10kΩ
5. LED Verde 7 + resistencia 220Ω
6. LED Rojo 8 + resistencia 220Ω
7. LED Amarillo 12 + resistencia 220Ω

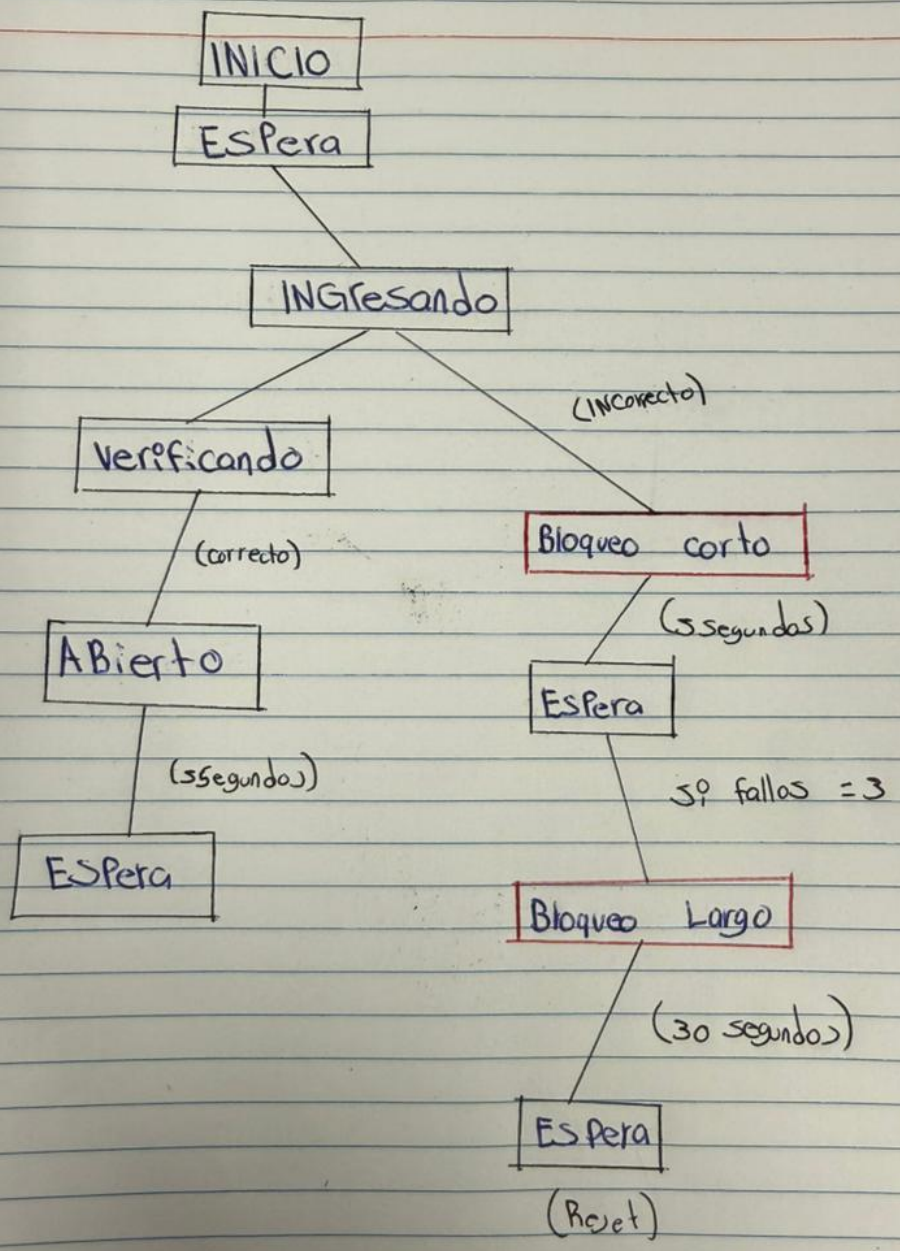
Lista De Materiales

1. 4 Resistencias 10k
2. 1 Protoboar
3. Jumpers
4. 1 Buzzer
5. 4 púlsadores
6. 3 Leds
7. 1 Arduino Uno
8. 1 Servomotor 9 PWM

Diagrama De Estados

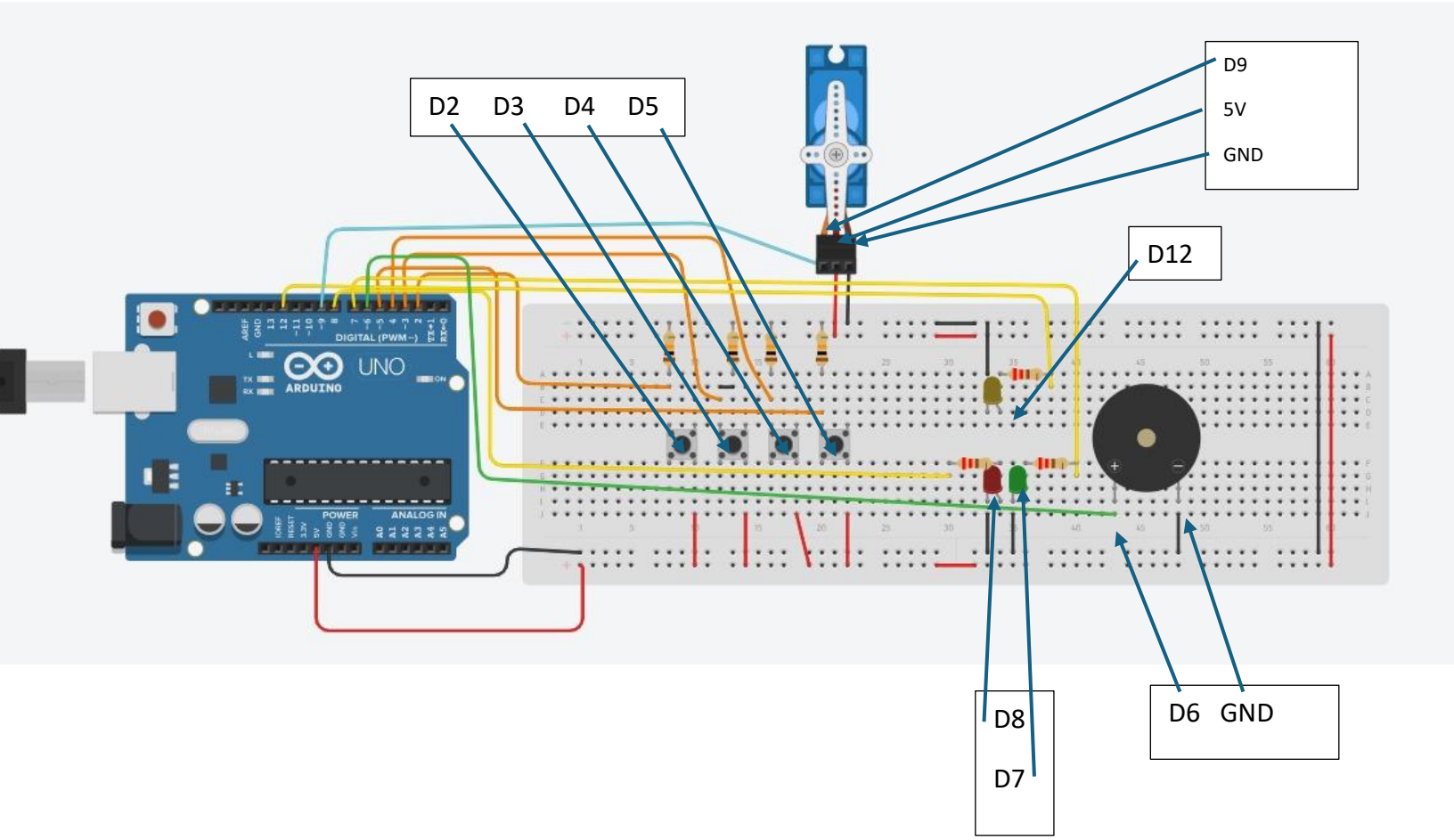
1. ESPERA
2. INGRESANDO
3. VERIFICANDO
4. ABIERTO
5. BLOQUEADO_CORTO
6. BLOQUEADO_LARGO

TRANSICIONES



ETAPA 2	Estudiante 3 — Diseñador de Circuito
Tareas a realizar	• Diseñar conexión de 4 pulsadores con pull-down externos • Diseñar conexión del servomotor (señal PWM, VCC, GND) • Conectar 3 LEDs con resistencias en pines digitales • Conectar buzzer en pin PWM • Verificar que el servo NO comparte pin con buzzer (ambos necesitan tone/PWM)
Entregables	• **Esquema de conexión completo con etiquetas de pines** • **Nota de advertencia: servo en pin 9, buzzer en pin 6 (no compartir)**
Criterios de aceptación	• Los 4 pulsadores tienen resistencias pull-down • El servomotor está en pin PWM diferente al buzzer • Los LEDs tienen resistencias limitadoras correctas

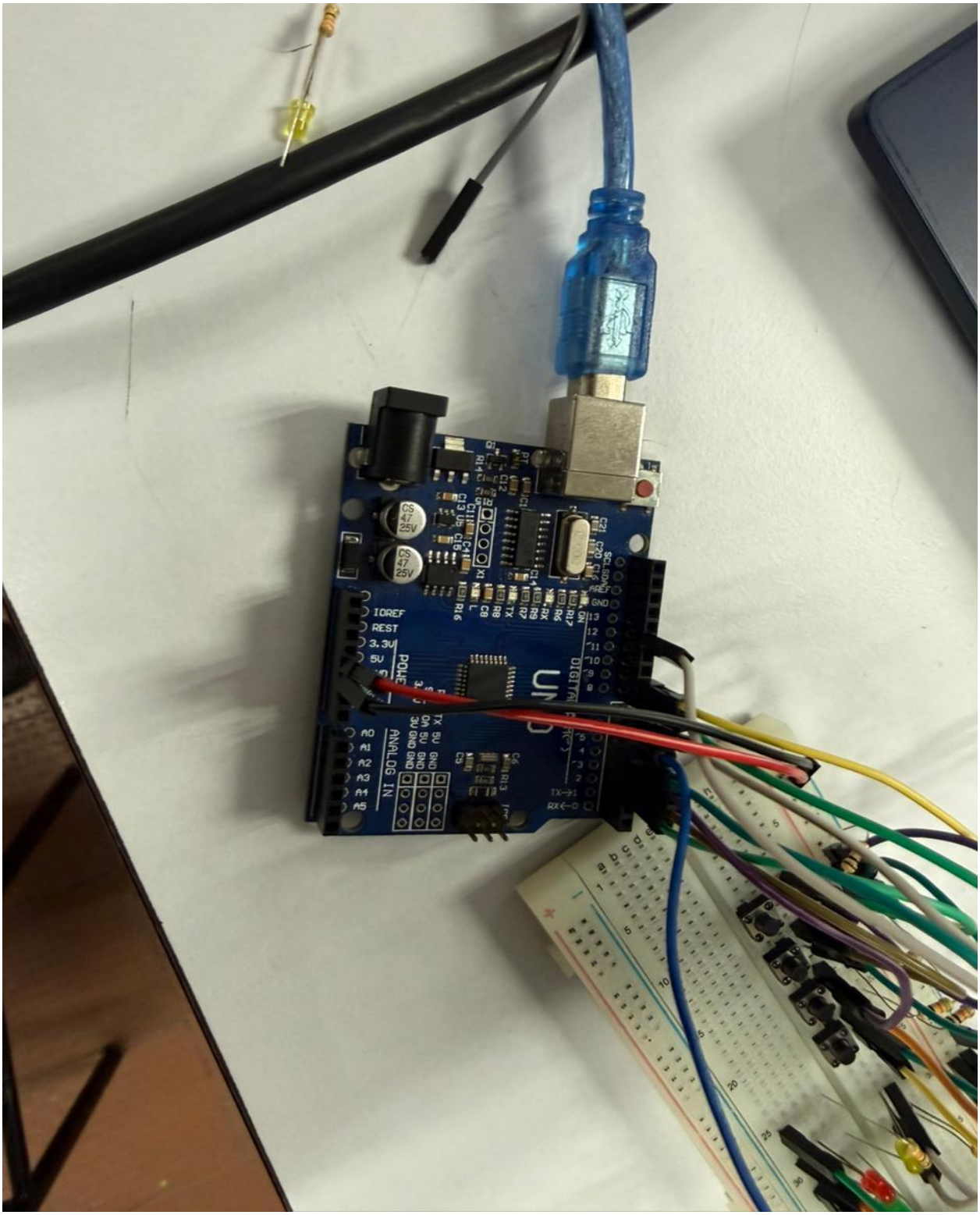
Esquema De Conexión

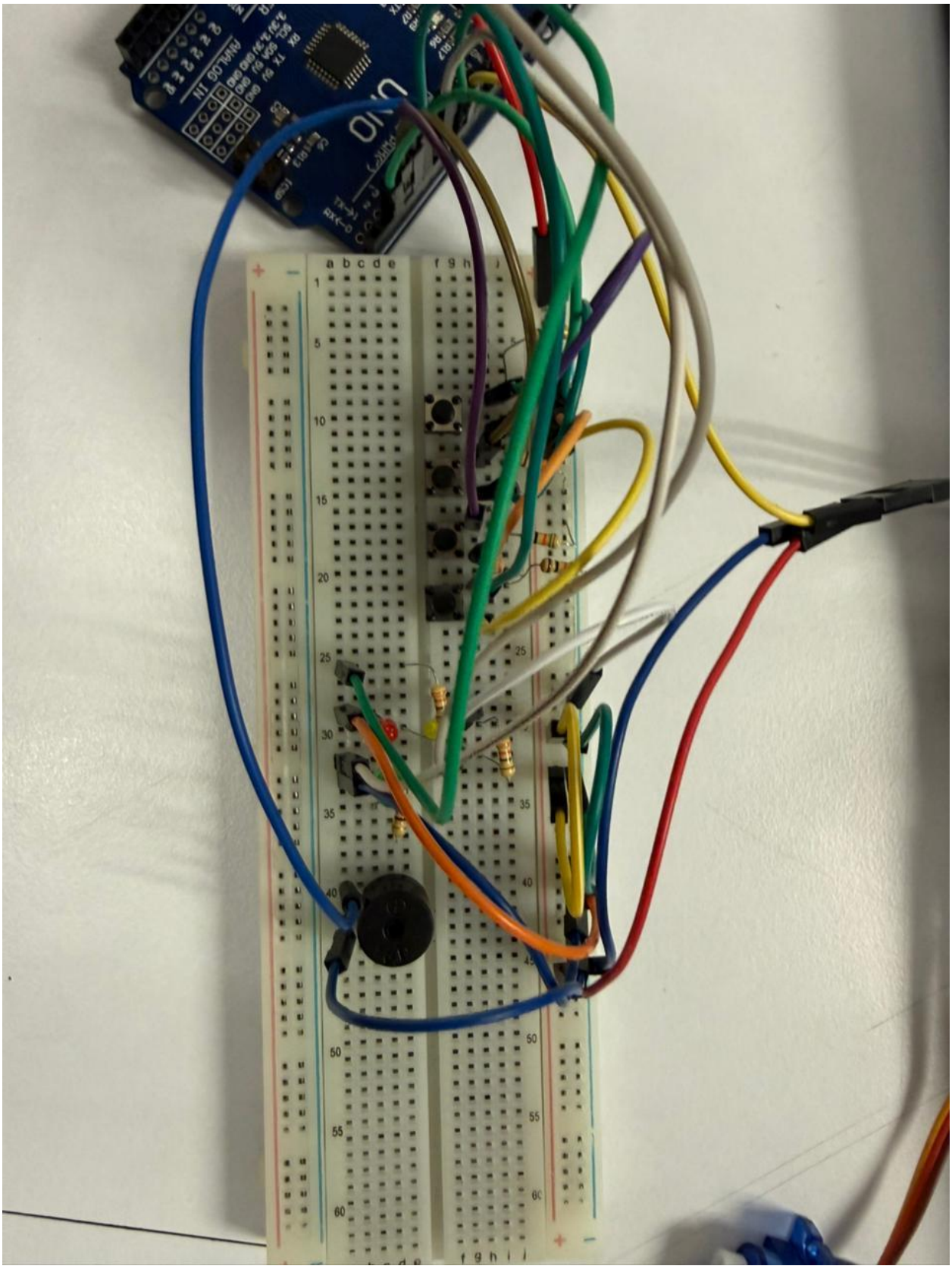


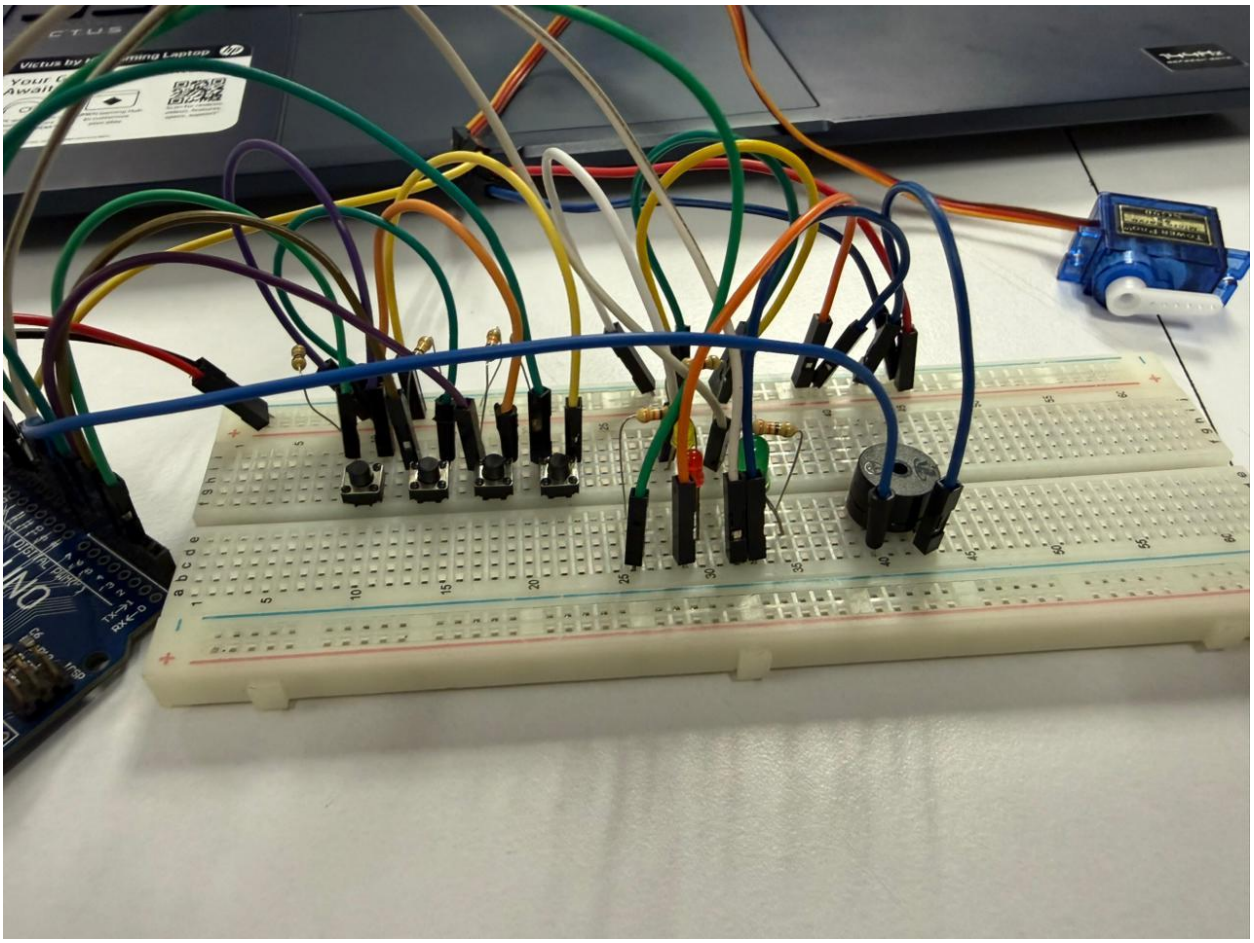
ETAPA 3	Estudiante 4 — Montador de Hardware
Tareas a realizar	• Montar los 4 pulsadores con sus resistencias pull-down • Conectar el servomotor con cables de colores correctos (naranja=señal, rojo=VCC, marrón/negro=GND)

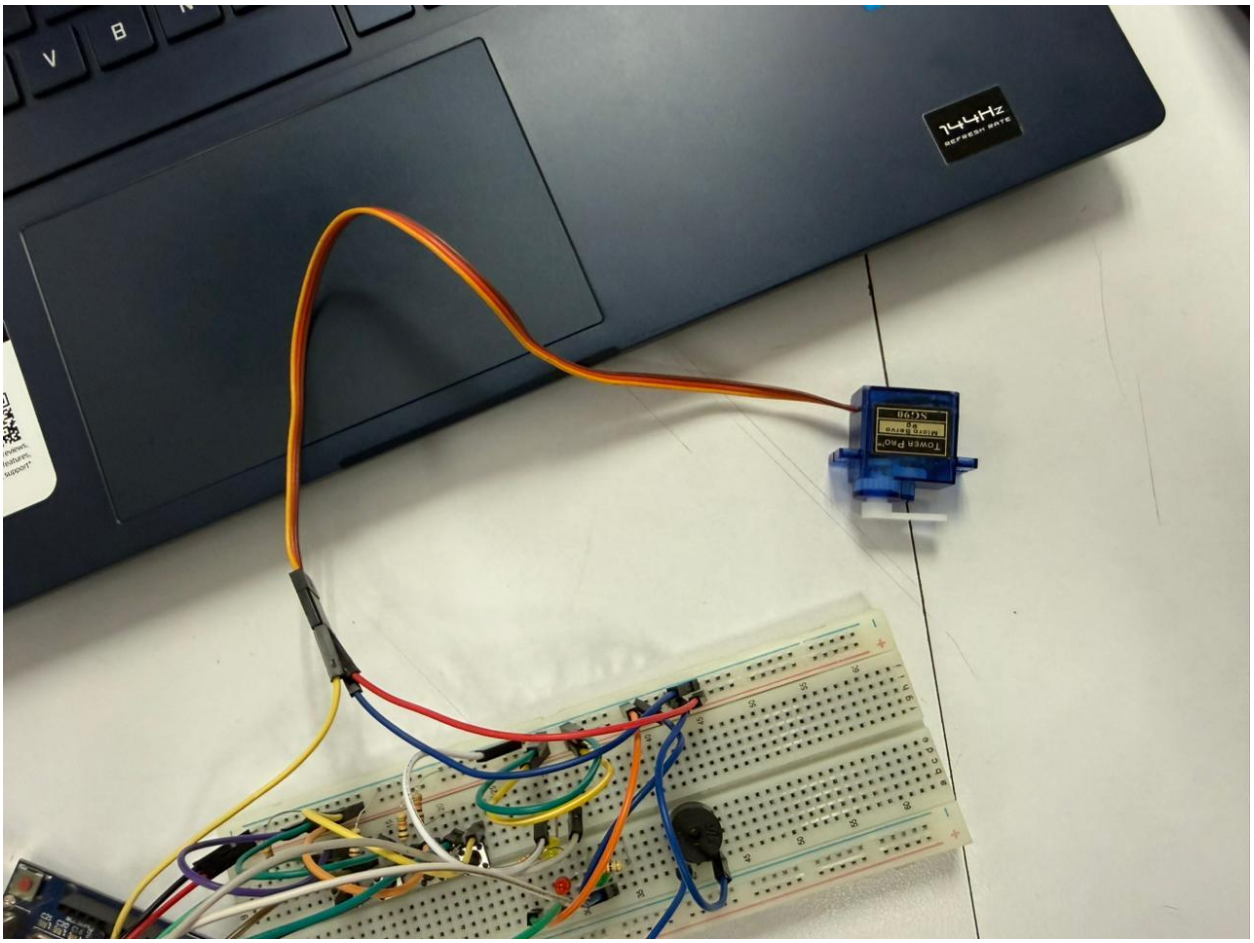
	• Montar 3 LEDs con resistencias • Montar buzzer • Ordenar los cables para legibilidad y fotografiar
Entregables	• **Circuito montado y fotografiado** • **Nota sobre colores de cables del servo**
Criterios de aceptación	• El servo está correctamente alimentado (5V) • Los 4 botones están en pines digitales correctos • El circuito está limpio y organizado

Circuito Montado









ETAPA 4	Estudiante 5 — Programador
Tareas a realizar	• Incluir librería Servo.h • Programar lectura de 4 pulsadores con debounce individual (millis()) • Implementar la máquina de estados: ESPERA→INGRESANDO→VERIFICANDO→ABIERTO/BLOQUEADO • Implementar contador de intentos fallidos con reset al éxito • Programar servo (attach, write, detach) • Programar buzzer con tonos diferenciados por resultado • Programar temporización de bloqueos con millis()
Entregables	• **Sketch con máquina de estados comentada** • **El código correcto debe estar como constante o array en el programa**
Criterios de aceptación	• La máquina de estados es correcta y estable • El bloqueo de 30 segundos funciona al 3er fallo • El servo regresa automáticamente a 0° después de 5 segundos

SKETCH

```
#include <Servo.h>

// Pines
#define BTN1 2
#define BTN2 3
#define BTN3 4
#define BTN4 5
#define LED_VERDE 7
#define LED_ROJO 8
#define LED_AMARILLO 12
#define BUZZER 6
#define SERVO_PIN 9

// Codigo correcto
const int codigoCorrecto[4] = {1, 2, 3, 4};

// Estados
enum Estado { ESPERA, INGRESANDO, VERIFICANDO, ABIERTO,
BLOQUEADO_CORTO, BLOQUEADO_LARGO };
Estado estado = ESPERA;

int intento[4];
int indice = 0;
int fallosConsecutivos = 0;
int intentoNumero = 0;
```

```
unsigned long tiempoInicio = 0;
const int TIEMPO_ABIERTO = 5000;
const int TIEMPO_BLOQUEO_CORTO = 5000;
const int TIEMPO_BLOQUEO_LARGO = 30000;

Servo servo;

//  Control de flanco
bool estadoAnterior[4] = {false, false, false, false};

bool leerBoton(int pin, int idx) {
 bool lectura = digitalRead(pin);

// Detecta solo cuando presiona
if (lectura && !estadoAnterior[idx]) {
estadoAnterior[idx] = true;
return true;
}

if (!lectura) {
estadoAnterior[idx] = false;
}

return false;
}

void setup() {
```

```
Serial.begin(9600);

pinMode(BTN1, INPUT);
pinMode(BTN2, INPUT);
pinMode(BTN3, INPUT);
pinMode(BTN4, INPUT);

pinMode(LED_VERDE, OUTPUT);
pinMode(LED_ROJO, OUTPUT);
pinMode(LED_AMARILLO, OUTPUT);
pinMode(BUZZER, OUTPUT);

servo.attach(SERVO_PIN);
servo.write(0);

Serial.println("Sistema listo");
}

void loop() {

switch(estado) {

case ESPERA:
indice = 0;
for(int i=0;i<4;i++) intento[i]=0;

digitalWrite(LED_ROJO, LOW);
```

```
digitalWrite(LED_VERDE, LOW);

estado = INGRESANDO;
break;

case INGRESANDO:
// Parpadeo led amarillo
static unsigned long lastBlink = 0;
if (millis() - lastBlink > 300) {
digitalWrite(LED_AMARILLO, !digitalRead(LED_AMARILLO));
lastBlink = millis();
}

if (leerBoton(BTN1,0)) { intento[indice]=1; indice++; }
else if (leerBoton(BTN2,1)) { intento[indice]=2; indice++; }
else if (leerBoton(BTN3,2)) { intento[indice]=3; indice++; }
else if (leerBoton(BTN4,3)) { intento[indice]=4; indice++; }

if (indice == 4) estado = VERIFICANDO;
break;

case VERIFICANDO: {
bool ok = true;

for(int i=0;i<4;i++){
if(intento[i] != codigoCorrecto[i]) ok = false;
}
```

```
intentoNumero++;

Serial.print("Intento ");
Serial.print(intentoNumero);
Serial.print(": ");

for(int i=0;i<4;i++) Serial.print(intento[i]);

if (ok) {
Serial.println(" -> CORRECTO");

fallosConsecutivos = 0;
estado = ABIERTO;
tiempoInicio = millis();

servo.write(90);
digitalWrite(LED_VERDE, HIGH);

tone(BUZZER, 1000, 200);
delay(250);
tone(BUZZER, 1500, 200);

} else {
Serial.println(" -> INCORRECTO");

fallosConsecutivos++;
```

```
digitalWrite(LED_ROJO, HIGH);

for(int i=0;i<3;i++){
tone(BUZZER, 300, 200);
delay(300);
}
noTone(BUZZER);

tiempoInicio = millis();

if (fallosConsecutivos >= 3) {
estado = BLOQUEADO_LARGO;
} else {
estado = BLOQUEADO_CORTO;
}
}
} break;

case ABIERTO:
if (millis() - tiempoInicio >= TIEMPO_ABIERTO) {
servo.write(0);
digitalWrite(LED_VERDE, LOW);
estado = ESPERA;
}
break;

case BLOQUEADO_CORTO:
```

```
if (millis() - tiempoInicio >= TIEMPO_BLOQUEO_CORTO) {
digitalWrite(LED_ROJO, LOW);
estado = ESPERA;
}
break;

case BLOQUEADO_LARGO:
digitalWrite(LED_ROJO, HIGH);

if (millis() - tiempoInicio >= TIEMPO_BLOQUEO_LARGO) {
digitalWrite(LED_ROJO, LOW);
fallosConsecutivos = 0;
estado = ESPERA;
}
break;

}
}
```

ETAPA 5	Estudiante 6 — Tester / QA
Tareas a realizar	• Probar REQ-01/02: ingresar código correcto y verificar apertura • Probar REQ-03: medir tiempo de apertura (5 segundos) y cierre automático • Probar REQ-04: ingresar código incorrecto y verificar bloqueo 5s • Probar REQ-05: verificar parpadeo LED amarillo durante ingreso • Probar REQ-06: ingresar 3 veces incorrecto y verificar bloqueo 30s • Probar REQ-07: verificar formato del Serial Monitor • Probar caso borde: ingresar solo 3 dígitos y esperar
Entregables	• **Reporte de pruebas con PASS/FAIL** • **Descripción del comportamiento en caso borde (3 dígitos sin completar)**
Criterios de aceptación	• El bloqueo de 30s es verificado con cronómetro • El caso borde está documentado aunque no esté en requisitos

Al ingreso de solo 3 dígitos

Al presionaros no hace ningún movimiento y se queda esperando hasta ser ingresado un 4 digito el programa acepta su código hasta completar los 4 dígitos de lo contrario se quedara esperando, al ingresar los 4 dígitos no importa que tiempo tarde en ingresar cada uno de ellos el programa comparara el código final y si es correcto abrirá, de lo contrario no lo hará.

PRUEBAS

✅ 1. Probar REQ-01/02: ingresar código correcto y verificar apertura
✅ 2. Probar REQ-03: medir tiempo de apertura (5 segundos) y cierre automático
✅ 3. Probar REQ-04: ingresar código incorrecto y verificar bloqueo 5s
✅ 4. Probar REQ-05: verificar parpadeo LED amarillo durante ingreso
✅ 5. Probar REQ-06: ingresar 3 veces incorrecto y verificar bloqueo 30s
✅ 6. Probar REQ-07: verificar formato del Serial Monitor
✅ 7. Probar caso borde: ingresar solo 3 dígitos y esperar

Conclusiones

El sistema presenta un funcionamiento correcto en la mayoría de las pruebas realizadas, cumpliendo adecuadamente con la lógica de control de acceso propuesta. Sin embargo, se identifican algunas áreas de mejora en términos de seguridad.

Una posible optimización sería implementar un límite de tiempo por cada dígito ingresado, lo que evitaría intentos excesivamente lentos o manipulaciones deliberadas del sistema. Asimismo, el mecanismo de bloqueo podría fortalecerse mediante un incremento progresivo del tiempo de espera conforme aumenten los intentos fallidos, en lugar de mantener únicamente un bloqueo fijo de 30 segundos. Esto reduciría la posibilidad de realizar múltiples combinaciones en un corto periodo de tiempo.

A pesar de estas observaciones, el sistema resulta funcional y eficiente para aplicaciones prácticas básicas, siendo suficiente en contextos donde no se requiera un alto nivel de seguridad. Su funcionamiento es similar al de sistemas de acceso reales, como alarmas o cerraduras electrónicas, aunque estos suelen incorporar claves de mayor longitud y algoritmos de protección más avanzados.